

Layer	Pythia on S3 (PAA)		Pythia on S3 (AA)	
	State Probe	Parity Probe	State Probe	Parity Probe
0	4.16×10^{-6}	3.03×10^{-6}	6.33×10^{-6}	2.71×10^{-6}
1	3.36×10^{-6}	3.31×10^{-6}	2.62×10^{-6}	2.82×10^{-6}
2	4.52×10^{-6}	5.45×10^{-6}	1.46×10^{-6}	8.97×10^{-7}
3	4.31×10^{-5}	5.59×10^{-4}	5.00×10^{-6}	1.38×10^{-6}
4	1.58×10^{-5}	3.72×10^{-5}	4.16×10^{-6}	1.11×10^{-6}
5	1.70×10^{-5}	5.38×10^{-6}	1.97×10^{-6}	1.11×10^{-6}
6	1.87×10^{-5}	7.02×10^{-6}	4.73×10^{-6}	1.59×10^{-6}
7	1.29×10^{-5}	1.34×10^{-5}	7.79×10^{-6}	3.05×10^{-6}
8	3.18×10^{-5}	3.08×10^{-5}	1.18×10^{-5}	2.64×10^{-6}
9	2.96×10^{-4}	6.85×10^{-5}	2.36×10^{-5}	3.53×10^{-6}
10	3.10×10^{-4}	7.98×10^{-5}	2.60×10^{-5}	2.96×10^{-6}
11	1.86×10^{-4}	8.22×10^{-5}	8.83×10^{-6}	2.77×10^{-6}
12	7.05×10^{-5}	4.01×10^{-5}	1.20×10^{-6}	2.75×10^{-6}

Table 1. Standard deviations across probe accuracies. We focus our analysis on S3 Pythia models and train 10 probes on different subsets of the S3 dataset. Standard deviations are tiny in all cases, indicating robust signatures.

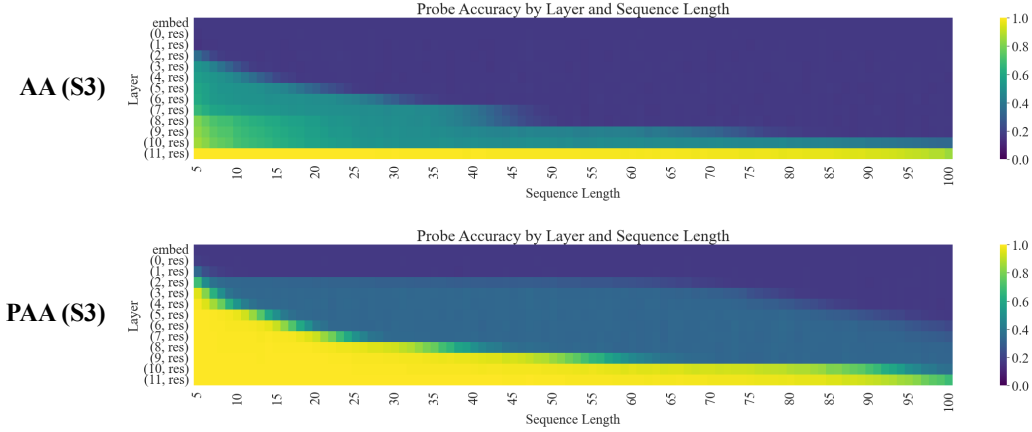


Figure 10. We plot the average accuracy of a linear probe trained to predict the final state of an action sequence A , given the corresponding final-token hidden representation of AA and PAA models on A . We find that both types of models can handle longer sequence lengths as we go down the network, and that PAA models compute the parities of sequences at roughly layer 2, after which they can get the parity of the state correct but not the exact state.

final-layer hidden representations from an AA model that *does* linearly encode the parity (from-scratch GPT2-base on S_3). When projected onto three components that explain 49.9% of the variance in the data, we find a clear linear separation between the odd and even parity representations.

D. Full Linear Decomposition Results

D.1. S_3

We visualize the linear decomposition of the last-layer or penultimate-layer representations across various PAA models. We find the triangular prism shape similar to Figure 4 in all of them, but there was no consistency in which states were paired to form the clusters.

One interpretation is that PAA models may be learning various presentations of S_3 , with each clustering configuration corresponding to a different presentation. Generally speaking, S_n can be generated by a 2-cycle and an n -cycle: any permutation of S_n can be created by composing these two permutations. For example, S_3 can be generated by the 2-cycle $1 \leftrightarrow 2$ and 3-cycle $1 \rightarrow 2 \rightarrow 3 \rightarrow 1$, which corresponds to the clustering $\{(123, 213), (312, 132), (231, 321)\}$: the states

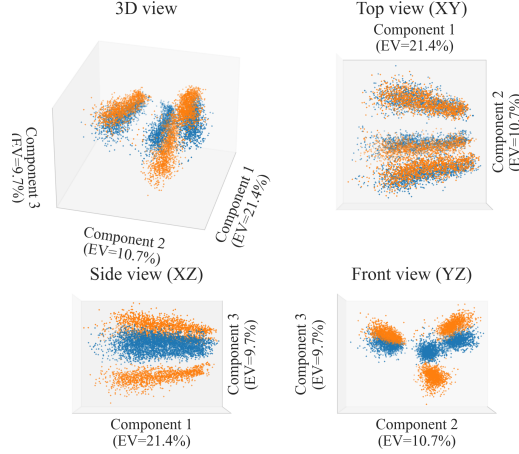


Figure 11. Example activations from an AA model that *does not* linearly encode parity at the final layer, projected on three principal components with a total explained variance of 41.8%. Blue points have even parity, while orange points have odd parity.

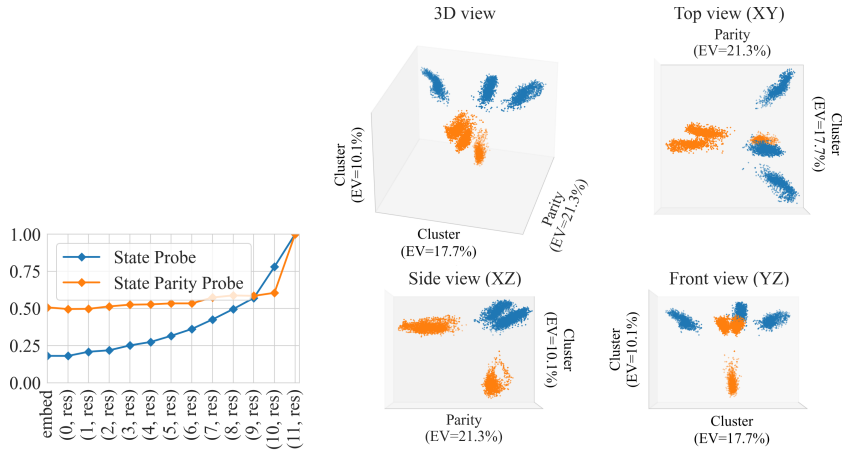


Figure 12. Example activations from an AA model that *does* linearly encode parity at the final layer. Blue points have even parity, while orange points have odd parity. (Left) State and state parity probe signatures of this model. (Right) projection of hidden representations onto three components with a total explained variance of 49.9%.

within the cluster can be transformed into each other by applying $1 \leftrightarrow 2$, while states between clusters are related to each other by $1 \rightarrow 2 \rightarrow 3 \rightarrow 1$. PAA models that cluster according to this pattern may have learned these generators.

D.2. S_5

What happens in models that learn PAA in S_5 ? We visualize the penultimate-layer representation of a Pythia-160M model that learned PAA on S_5 in 3D space, with parity along one axis and two orthogonal directions along the other two. We find 4 distinct clusters, corresponding to the position of 1 in the state (states having 1 in position 4 and 1 in position 5 are clustered together).

E. Simulating State Tracking in Natural Language

To emulate a more practical scenario, we train pre-trained and from-scratch Pythia models on a version of the S3 permutation composition task expressed in *natural language*. For example, the permutation “132” would be expressed as “swap positions 2 and 3,” while “312” would be “rotate the last item to the front.” We train LLMs to predict the final state (e.g., 231) from the final *period* token of the sequence. For example, the following sequence:

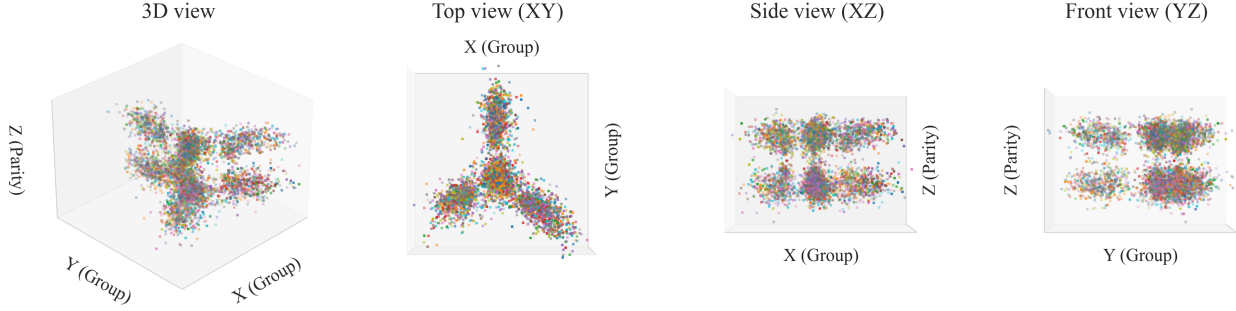


Figure 13. Projecting models that learn PAA on S_5 into 3D space. Unlike PAA models on S_3 (Figure 4), the state cannot be fully represented by a clean decomposition into 3 directions (notice the colors superimposed on each other). However, we do still find symmetry across the parity axis, similar to S_3 . Moreover, there are 4 neat clusters, one at the center, and three outward protruding “prongs”. We find that the clusters correspond to the position of 1 in the state.

Swap positions 2 and 3. Rotate the last item to the front. Swap positions 1 and 2.

would map to permutation sequence “132, 312, 213” and finally correspond to the state “123” after the swaps. We then conduct a similar style of probing and activation patching experiments on models trained on this task.

E.1. Probing experiments

We train probes to map from the activation of each layer at the position of the final “.” token to the final state. Shown in Figure 14, as in our results on synthetic data, probing results are consistent with associative mechanisms – the state probe improves exponentially over layers. In pre-trained Pythia, we see the model represent the state parity much earlier (in terms of layer) than the actual state representation, a signature of the PAA algorithm. For non-pre-trained Pythia, the accuracy of the state and state parity probes increases at a similar rate, indicating that it is more likely learning an associative algorithm; whether it is the strict AA algorithm we identified in the synthetic case is unclear.

E.2. Activation patching experiments

We patch prefixes up to a fixed token position.⁸ As shown in Figure 14, both models display a distinctly associative signature with exponentially longer prefixes being disregarded for the final state prediction as depth increases. Furthermore, the pre-trained Pythia model possesses a light blue middle section – a sign of the PAA algorithm. Interestingly, the pre-trained Pythia results are significantly more “compressed” over the layers – the LLM computes the state very early on. We suspect this may be due to the pre-trained LLM taking advantage of its innate natural language understanding (and perhaps pre-trained state tracking abilities!) to quickly solve the task in an early layer.

F. Full Attention Heads Analysis

F.1. Formalizing Parity Heads

To formalize a metric for whether an attention head behaves like a parity head, we define a *parity head score* as the percentage of sequence lengths (ranging from 5 to 80) over which the head places *significantly* more attention on odd-parity permutations than even-parity permutations, measuring significance using a 95% confidence interval.

Definition F.1. Let $\alpha_{i,\ell}^{(H)}(x)$ be the attention weight of the H th attention head in layer ℓ at position i for input x where x is the list of actions $[a_1 \dots a_t]$.

⁸Note that token positions may no longer be aligned in natural language: “swap” actions have 5 tokens, and “rotate” actions have 7. Thus, we may not be replacing the activations of the same number of actions between prompts. Nonetheless, we still believe the activation patching results serve as a good proxy for estimating how information gets propagated through the layers.